Chapter 12. Transactions

All of the examples thus far in this book have been individual, independent SQL statements. While this may be the norm for ad hoc reporting or data maintenance scripts, application logic will frequently include multiple SQL statements that need to execute together as a logical unit of work. This chapter explores *transactions*, which are the mechanism used to group a set of SQL statements together such that either all or none of the statements succeed.

Multiuser Databases

Database management systems allow a single user to query and modify data, but in today's world there may be thousands of people making changes to a database simultaneously. If every user is only executing queries, such as might be the case with a data warehouse during normal business hours, then there are very few issues for the database server to deal with. If some of the users are adding and/or modifying data, however, the server must handle quite a bit more bookkeeping.

Let's say, for example, that you are running a report that sums up the current week's film rental activity. At the same time you are running the report, however, the following activities are occurring:

- A customer rents a film.
- A customer returns a film after the due date and pays a late fee.
- Five new films are added to inventory.

While your report is running, therefore, multiple users are modifying the underlying data, so what numbers should appear on the report? The answer depends somewhat on how your server handles *locking*, which is described in the next section.

Locking

Locks are the mechanism the database server uses to control simultaneous use of data resources. When some portion of the database is locked, any other users wishing to

modify (or possibly read) that data must wait until the lock has been released. Most database servers use one of two locking strategies:

- Database writers must request and receive from the server a *write lock* to modify data, and database readers must request and receive from the server a *read lock* to query data. While multiple users can read data simultaneously, only one write lock is given out at a time for each table (or portion thereof), and read requests are blocked until the write lock is released.
- Database writers must request and receive from the server a write lock to modify data, but readers do not need any type of lock to query data. Instead, the server ensures that a reader sees a consistent view of the data (the data seems the same even though other users may be making modifications) from the time her query begins until her query has finished. This approach is known as *versioning*.

There are pros and cons to both approaches. The first approach can lead to long wait times if there are many concurrent read and write requests, and the second approach can be problematic if there are long-running queries while data is being modified. Of the three servers discussed in this book, Microsoft SQL Server uses the first approach, Oracle Database uses the second approach, and MySQL uses both approaches (depending on your choice of *storage engine*, which we'll discuss a bit later in the chapter).

Lock Granularities

There are also a number of different strategies that you may employ when deciding *how* to lock a resource. The server may apply a lock at one of three different levels, or *granularities*:

Table locks

Keep multiple users from modifying data in the same table simultaneously

Page locks

Keep multiple users from modifying data on the same page (a page is a segment of memory generally in the range of 2 KB to 16 KB) of a table simultaneously

Row locks

Keep multiple users from modifying the same row in a table simultaneously

Again, there are pros and cons to these approaches. It takes very little bookkeeping to lock entire tables, but this approach quickly yields unacceptable wait times as the num-

ber of users increases. On the other hand, row locking takes quite a bit more bookkeeping, but it allows many users to modify the same table as long as they are interested in different rows. Of the three servers discussed in this book, Microsoft SQL Server uses page, row, and table locking, Oracle Database uses only row locking, and MySQL uses table, page, or row locking (depending, again, on your choice of storage engine). SQL Server will, under certain circumstances, *escalate* locks from row to page, and from page to table, whereas Oracle Database will never escalate locks.

To get back to your report, the data that appears on the pages of the report will mirror either the state of the database when your report started (if your server uses a versioning approach) or the state of the database when the server issues the reporting application a read lock (if your server uses both read and write locks).

What Is a Transaction?

If database servers enjoyed 100% uptime, if users always allowed programs to finish executing, and if applications always completed without encountering fatal errors that halt execution, then there would be nothing left to discuss regarding concurrent database access. However, we can rely on none of these things, so one more element is necessary to allow multiple users to access the same data.

This extra piece of the concurrency puzzle is the *transaction*, which is a device for grouping together multiple SQL statements such that either *all* or *none* of the statements succeed (a property known as *atomicity*). If you attempt to transfer $500 from your savings account to your checking account, you would be a bit upset if the money were successfully withdrawn from your savings account but never made it to your checking account. Whatever the reason for the failure (the server was shut down for maintenance, the request for a page lock on the `account` table timed out, etc.), you want your $500 back.

To protect against this kind of error, the program that handles your transfer request would first begin a transaction, then issue the SQL statements needed to move the money from your savings to your checking account, and, if everything succeeds, end the transaction by issuing the `commit` command. If something unexpected happens, however, the program would issue a `rollback` command, which instructs the server to undo all changes made since the transaction began. The entire process might look something like the following:

```
START TRANSACTION;
```

```
 /* withdraw money from first account, making sure balance is sufficient */
UPDATE account SET avail_balance = avail_balance - 500
WHERE account_id = 9988
  AND avail_balance > 500;

IF <exactly one row was updated by the previous statement> THEN
  /* deposit money into second account */
  UPDATE account SET avail_balance = avail_balance + 500
    WHERE account_id = 9989;

  IF <exactly one row was updated by the previous statement> THEN
    /* everything worked, make the changes permanent */
    COMMIT;
  ELSE
    /* something went wrong, undo all changes in this transaction */
    ROLLBACK;
  END IF;
ELSE
  /* insufficient funds, or error encountered during update */
  ROLLBACK;
END IF;
```

NOTE

While the previous code block may look similar to one of the procedural languages provided by the major database companies, such as Oracle's PL/SQL or Microsoft's Transact-SQL, it is written in pseudocode and does not attempt to mimic any particular language.

The previous code block begins by starting a transaction and then attempts to remove $500 from the checking account and add it to the savings account. If all goes well, the transaction is committed; if anything goes awry, however, the transaction is rolled back, meaning that all data changes since the beginning of the transaction are undone.

By using a transaction, the program ensures that your $500 either stays in your savings account or moves to your checking account, without the possibility of it falling into a crack. Regardless of whether the transaction was committed or was rolled back, all resources acquired (e.g., write locks) during the execution of the transaction are released when the transaction completes.

Of course, if the program manages to complete both `update` statements but the server shuts down before a `commit` or `rollback` can be executed, then the transaction will be rolled back when the server comes back online. (One of the tasks that a database server

must complete before coming online is to find any incomplete transactions that were underway when the server shut down and roll them back.) Additionally, if your program finishes a transaction and issues a `commit` but the server shuts down before the changes have been applied to permanent storage (i.e., the modified data is sitting in memory but has not been flushed to disk), then the database server must reapply the changes from your transaction when the server is restarted (a property known as durability).

Starting a Transaction

Database servers handle transaction creation in one of two ways:

- An active transaction is always associated with a database session, so there is no need or method to explicitly begin a transaction. When the current transaction ends, the server automatically begins a new transaction for your session.
- Unless you explicitly begin a transaction, individual SQL statements are automatically committed independently of one another. To begin a transaction, you must first issue a command.

Of the three servers, Oracle Database takes the first approach, while Microsoft SQL Server and MySQL take the second approach. One of the advantages of Oracle's approach to transactions is that, even if you are issuing only a single SQL command, you have the ability to roll back the changes if you don't like the outcome or if you change your mind. Thus, if you forget to add a `where` clause to your `delete` statement, you will have the opportunity to undo the damage (assuming you've had your morning coffee and realize that you didn't mean to delete all 125,000 rows in your table). With MySQL and SQL Server, however, once you press the Enter key, the changes brought about by your SQL statement will be permanent (unless your DBA can retrieve the original data from a backup or from some other means).

The SQL:2003 standard includes a `start transaction` command to be used when you want to explicitly begin a transaction. While MySQL conforms to the standard, SQL Server users must instead issue the command `begin transaction`. With both servers, until you explicitly begin a transaction, you are in what is known as *autocommit mode*, which means that individual statements are automatically committed by the server. You can, therefore, decide that you want to be in a transaction and issue a start/begin transaction command, or you can simply let the server commit individual statements.

Both MySQL and SQL Server allow you to turn off autocommit mode for individual sessions, in which case the servers will act just like Oracle Database regarding transactions.

With SQL Server, you issue the following command to disable autocommit mode:

```
SET IMPLICIT_TRANSACTIONS ON
```

MySQL allows you to disable autocommit mode via the following:

```
SET AUTOCOMMIT=0
```

Once you have left autocommit mode, all SQL commands take place within the scope of a transaction and must be explicitly committed or rolled back.

NOTE

A word of advice: shut off autocommit mode each time you log in, and get in the habit of running all of your SQL statements within a transaction. If nothing else, it may save you the embarrassment of having to ask your DBA to reconstruct data that you have inadvertently deleted.

Ending a Transaction

Once a transaction has begun, whether explicitly via the `start transaction` command or implicitly by the database server, you must explicitly end your transaction for your changes to become permanent. You do this by way of the `commit` command, which instructs the server to mark the changes as permanent and release any resources (i.e., page or row locks) used during the transaction.

If you decide that you want to undo all the changes made since starting the transaction, you must issue the `rollback` command, which instructs the server to return the data to its pre-transaction state. After the `rollback` has been completed, any resources used by your session are released.

Along with issuing either the `commit` or `rollback` command, there are several other scenarios by which your transaction can end, either as an indirect result of your actions or as a result of something outside your control:

- The server shuts down, in which case your transaction will be rolled back automatically when the server is restarted.
- You issue an SQL schema statement, such as `alter table`, which will cause the current transaction to be committed and a new transaction to be started.

- You issue another `start transaction` command, which will cause the previous transaction to be committed.
- The server prematurely ends your transaction because the server detects a *deadlock* and decides that your transaction is the culprit. In this case, the transaction will be rolled back, and you will receive an error message.

Of these four scenarios, the first and third are fairly straightforward, but the other two merit some discussion. As far as the second scenario is concerned, alterations to a database, whether it be the addition of a new table or index or the removal of a column from a table, cannot be rolled back, so commands that alter your schema must take place outside a transaction. If a transaction is currently underway, therefore, the server will commit your current transaction, execute the SQL schema statement command(s), and then automatically start a new transaction for your session. The server will not inform you of what has happened, so you should be careful that the statements that comprise a unit of work are not inadvertently broken up into multiple transactions by the server.

The fourth scenario deals with deadlock detection. A deadlock occurs when two different transactions are waiting for resources that the other transaction currently holds. For example, transaction A might have just updated the `account` table and is waiting for a write lock on the `transaction` table, while transaction B has inserted a row into the `transaction` table and is waiting for a write lock on the `account` table. If both transactions happen to be modifying the same page or row (depending on the lock granularity in use by the database server), then they will each wait forever for the other transaction to finish and free up the needed resource. Database servers must always be on the lookout for these situations so that throughput doesn't grind to a halt; when a deadlock is detected, one of the transactions is chosen (either arbitrarily or by some criteria) to be rolled back so that the other transaction may proceed. Most of the time, the terminated transaction can be restarted and will succeed without encountering another deadlock situation.

Unlike the second scenario discussed earlier, the database server will raise an error to inform you that your transaction has been rolled back due to deadlock detection. With MySQL, for example, you will receive error 1213, which carries the following message:

> **Message: Deadlock found when trying to get lock; try restarting transaction**

As the error message suggests, it is a reasonable practice to retry a transaction that has been rolled back due to deadlock detection. However, if deadlocks become fairly common, then you may need to modify the applications that access the database to decrease

the probability of deadlocks (one common strategy is to ensure that data resources are always accessed in the same order, such as always modifying account data before inserting transaction data).

Transaction Savepoints

In some cases, you may encounter an issue within a transaction that requires a rollback, but you may not want to undo *all* of the work that has transpired. For these situations, you can establish one or more *savepoints* within a transaction and use them to roll back to a particular location within your transaction rather than rolling all the way back to the start of the transaction.

CHOOSING A STORAGE ENGINE

When using Oracle Database or Microsoft SQL Server, a single set of code is responsible for low-level database operations, such as retrieving a particular row from a table based on primary key value. The MySQL server, however, has been designed so that multiple storage engines may be utilized to provide low-level database functionality, including resource locking and transaction management. As of version 8.0, MySQL includes the following storage engines:

MyISAM

A nontransactional engine employing table locking

MEMORY

A nontransactional engine used for in-memory tables

CSV

A transactional engine that stores data in comma-separated files

InnoDB

A transactional engine employing row-level locking

Merge

A specialty engine used to make multiple identical MyISAM tables appear as a single table (a.k.a. table partitioning)

Archive

A specialty engine used to store large amounts of unindexed data, mainly for archival purposes

Although you might think that you would be forced to choose a single storage engine for your database, MySQL is flexible enough to allow you to choose a storage engine on a table-by-table basis. For any tables that might take part in transactions, however, you should choose the InnoDB engine, which uses row-level locking and versioning to provide the highest level of concurrency across the different storage engines.

You may explicitly specify a storage engine when creating a table, or you can change an existing table to use a different engine. If you do not know what engine is assigned to a table, you can use the `show table` command, as demonstrated by the following:

```
mysql> show table status like 'customer' \G;
*************************** 1. row ***************************
           Name: customer
         Engine: InnoDB
        Version: 10
     Row_format: Dynamic
           Rows: 599
 Avg_row_length: 136
    Data_length: 81920
Max_data_length: 0
   Index_length: 49152
      Data_free: 0
 Auto_increment: 599
    Create_time: 2019-03-12 14:24:46
    Update_time: NULL
     Check_time: NULL
      Collation: utf8_general_ci
       Checksum: NULL
 Create_options:
        Comment:
1 row in set (0.16 sec)
```

Looking at the second item, you can see that the `customer` table is already using the InnoDB engine. If it were not, you could assign the InnoDB engine to the transaction table via the following command:

```
ALTER TABLE customer ENGINE = INNODB;
```

All savepoints must be given a name, which allows you to have multiple savepoints within a single transaction. To create a savepoint named `my_savepoint`, you can do the following:

```
SAVEPOINT my_savepoint;
```

To roll back to a particular savepoint, you simply issue the `rollback` command followed by the keywords `to savepoint` and the name of the savepoint, as in:

```
ROLLBACK TO SAVEPOINT my_savepoint;
```

Here's an example of how savepoints may be used:

```
START TRANSACTION;

UPDATE product
SET date_retired = CURRENT_TIMESTAMP()
WHERE product_cd = 'XYZ';

SAVEPOINT before_close_accounts;

UPDATE account
SET status = 'CLOSED', close_date = CURRENT_TIMESTAMP(),
 last_activity_date = CURRENT_TIMESTAMP()
WHERE product_cd = 'XYZ';

ROLLBACK TO SAVEPOINT before_close_accounts;
COMMIT;
```

The net effect of this transaction is that the mythical XYZ product is retired but none of the accounts are closed.

When using savepoints, remember the following:

- Despite the name, nothing is saved when you create a savepoint. You must eventually issue a `commit` if you want your transaction to be made permanent.
- If you issue a `rollback` without naming a savepoint, all savepoints within the transaction will be ignored, and the entire transaction will be undone.

If you are using SQL Server, you will need to use the proprietary command `save transaction` to create a savepoint and `rollback transaction` to roll back to a savepoint, with each command being followed by the savepoint name.

Test Your Knowledge

Test your understanding of transactions by working through the following exercise. When you're done, compare your solution with that in Appendix B.

Exercise 12-1

Generate a unit of work to transfer $50 from account 123 to account 789. You will need to insert two rows into the `transaction` table and update two rows in the `account` table.

Use the following table definitions/data:

```
                Account:
account_id      avail_balance last_activity_date
----------      ------------- ------------------
123             500           2019-07-10 20:53:27
789             75            2019-06-22 15:18:35

                Transaction:
txn_id          txn_date account_id    txn_type_cd   amount
--------- ------------   -----------    -----------   --------
1001            2019-05-15 123         C             500
1002            2019-06-01 789         C             75
```

Use `txn_type_cd = 'C'` to indicate a credit (addition), and use `txn_type_cd = 'D'` to indicate a debit (subtraction).

Previous chapter

< 11. Conditional Logic

Next chapter

13. Indexes and Constraints >